

TFG - XplicaVoz

Jorge Aured Zarzoso, Germán Bravo Quintián,
Rafael López Rodríguez, Iván Pastor Sacristán

20 de febrero de 2026

Índice

1. Introducción	1
2. Estado del arte	2
2.1. Características	2
2.2. Modelos de clasificación	3
2.2.1. Perceptrón multicapa	3
2.2.2. Fine-tuning Wav2Vec2	3
2.2.3. Random Forest	4
2.2.4. LightGBM	5
2.2.5. Support Vector Machines (SVM)	6
2.3. Modelos de explicación	9
3. Metodología	9
3.1. Dataset	9
3.2. Preprocesamiento	10
3.3. Extracción de características	10
3.4. Modelos de clasificación	10
3.5. Modelos de explicación	10

1. Introducción

En los últimos años hemos visto un aumento considerable en el uso de la IA para suplantar la identidad de alguien de forma maliciosa clonando su voz. Estos son los conocidos *deepfakes*. El reconocimiento de dichos deepfakes es de vital importancia, pues suponen una gran amenaza. Por otro lado, las técnicas de detección de *deepfakes* no han avanzado al mismo ritmo, y esto lo vemos en la gran cantidad de estafas que se producen de este tipo.

Además, las personas ya no somos capaces de diferenciar entre una voz real y una generada, como se ve en [? ?], donde hacen sendos estudios exponiendo a personas reales a diversos audios, algunos reales, algunos generados y algunos que clonan voces

reales. En dichos estudios se ve que los que son 100 % generados nos cuesta poco detectarlos, pero entre los reales y los clones no diferenciamos bien, y rondamos el 60 % de precisión clasificando dichas voces en reales o falsas. Por esto no podemos fiarnos más de nuestras capacidades a la hora de diferenciar cuando una voz es real o no.

En este texto se proponen varias técnicas que podrían ser útiles para la detección de audios generados por IA. Para esta tarea es fundamental sacar las características de la voz, así como gráficas, para poder entrenar los modelos con dichos datos. Entre los audios deben haber tanto reales como generados, y estar medianamente equilibrados, para que los modelos aprendan las diferencias, y así sepan clasificar un audio que nunca han visto.

Pero el rendimiento del modelo de clasificación no es lo único que nos interesa. Hoy en día los modelos de IA se han convertido en *cajas negras*, donde no sabemos cómo hace las predicciones, dando lugar a problemas de confianza de los humanos respecto de dichas predicciones, como es en el ámbito médico o ingenieril, donde las decisiones que tomen las IAs deben ser precisas, puesto que la vida de las personas puede estar en juego. Para solventar este problema ha surgido la IA explicable (XAI), la cual toma dichos modelos *caja negra* y da una explicación de qué características de los datos afectan más a la decisión del modelo. En esta investigación vamos a aplicar XAI para ver qué afecta más a la clasificación de los audios dentro de los distintos grupos que mencionamos antes.

2. Estado del arte

2.1. Características

Para poder clasificar los audios en reales y generados tenemos que pensar primero en qué datos necesitamos pasarle a los modelos para que aprendan. Podemos separar los datos que debemos pasarle a dichos modelos en 3 grupos.

1. Características. Podemos extraer características de la voz como el tono fundamental, los formantes, las pausas, y otros. Con estos datos los modelos pueden aprender las relaciones entre los dos grupos de audios.
2. Gráficas. Podemos extraer gráficas de la voz como la forma de onda, el espectrograma de MEL, y otros. Con estas gráficas las CNN pueden aprender los puntos clave que diferencian a los audios reales de los generados.
3. Audio crudo. Existen modelos llamados wav2vec los cuales aprenden directamente de los audios, sacando un vector denso de características latentes en los audios, para después hacer ajuste fino o *fine-tuning* de un modelo para clasificar los audios.

Con estos 3 enfoques vamos a clasificar los audios y veremos cuál funciona mejor.

2.2. Modelos de clasificación

2.2.1. Perceptrón multicapa

Hemos usado el código de extracción de características para procesar los audios que tenemos en el dataset personalizado con 100 audios en español (50 reales y 50 generados a partir de esos reales). Una vez preprocesados los audios con sus características extraídas, empleo una CNN (red neuronal convolucional) para sacar un embedding de 128 dimensiones del espectrograma del audio y un MLP (perceptrón multicapa) para sacar un embedding de 64 dimensiones de las características paralingüísticas del audio. Ahora las concateno teniendo un embedding de 192 dimensiones siendo las primeras 64 las características y las otras 128 las del espectrograma.

Después creamos un perceptrón multicapa cuyo objetivo es clasificar el audio en si es generado o es real empleando para ello lógica difusa. Para ello pasamos la salida del perceptrón por un controlador fuzzy que irá aprendiendo también sus parámetros y adaptará la salida para que esté entre 0 y 1 siendo que cuanto más cercano al 0, más posible es que el audio sea real y no sea generado. En la práctica, como el dataset es de 100 datos, no hay suficientes para entrenar un perceptrón de forma correcta ya que cuando hicimos las pruebas, todas las predicciones del perceptrón no se alejan del 0.5 que en este contexto de probabilidad continuo significa que el modelo está indeciso y no lo sabe clasificar como real o como generado.

2.2.2. Fine-tuning Wav2Vec2

Wav2Vec2 es un modelo de aprendizaje profundo para procesamiento de voz que permite obtener representaciones acústicas de alta calidad directamente desde la señal cruda de audio, sin necesidad de características manuales. Fue propuesto por Meta en 2020.

Este modelo recibe como entrada la forma de onda sin procesar del audio (audio crudo) y su salida debe ser decodificada utilizando Wav2Vec2CTCTokenizer.

El fine-tuning se hará con el dataset de 100 datos que tenemos para entrenar el Wav2Vec2 en la tarea de clasificación de audio. Al igual que con el modelo anterior, también lo pasaré por un controlador fuzzy para que nos diga cuan probable es que el audio sea real o IA.

El proceso comienza creando un dataset de entrenamiento y otro de prueba en el que cada uno tendrá el mismo número de audios reales y generados y una etiqueta que indique si es humano o no. Una vez tenemos el dataset, hay que preprocesar los audios ya que en un inicio solo tiene las rutas de los audios por lo que ahora hay que cargar los audios de verdad. Para esta tarea usaremos el propio extractor de características que tiene el modelo (Wav2Vec2FeatureExtractor) fijando la frecuencia de muestreo a 16KHz y activando el padding que rellenará los audios cortos con ceros y trucará los audios largos para que todos tengan la misma longitud. Este preprocesado resultará en que ahora en el dataset, en lugar de haber rutas a los audios, hay tensores de Torch representándolos.

Ahora que ya tenemos el dataset de entrenamiento preprocesado, lo volvemos a dividir en entrenamiento y validación y entrenamos el modelo ya preentrenado para la nueva tarea de detección de voz generada por ia. Para evaluar el modelo se emplearon tres métricas, la precisión (verdaderos positivos / (verdaderos positivos + falsos positivos), la f1 (Media armónica entre precisión y recall) y el AUC (Área bajo la curva: Capacidad del modelo para distinguir entre clases positivas y negativas). El fine-tuning ha sido muy leve ya que con los 90 audios (80 de entrenamiento y 10 de validación) hay que congelar el aprendizaje del modelo para evitar un sobreentrenamiento.

Una vez hecho el fine-tuning, hacemos el mismo preprocesado para el dataset de test y ponemos el modelo en modo evaluación. Los resultados que de el modelo se pasan por un clasificador fuzzy que se encargará del postprocesado de la salida que la normalizará entre 0 y 1 siendo que los audios cuya score está entre 0.4 y 0.6 no los sabe clasificar, los que están por debajo de 0.4 son audios reales y los que están por encima de 0.6 son audios generados.

2.2.3. Random Forest

Random Forest es un algoritmo de aprendizaje automático basado en la combinación de múltiples árboles de decisión, propuesto por Leo Breiman en 2001 [?]. Pertenecce a la familia de métodos *ensemble*, que mejoran el rendimiento agregando las predicciones de varios modelos más simples.

Arquitectura y funcionamiento

El algoritmo construye un “bosque” de árboles de decisión donde cada árbol se entrena de forma ligeramente diferente, introduciendo aleatoriedad controlada en dos niveles:

- **Bootstrap aggregating (bagging):** Cada árbol se entrena con una muestra aleatoria del conjunto de datos original, permitiendo repeticiones. Esto significa que algunos ejemplos aparecen varias veces en el entrenamiento de un árbol, mientras que otros (alrededor de un tercio) quedan fuera (*out-of-bag*) y sirven para validación interna.
- **Selección aleatoria de características:** En cada división de un nodo del árbol, solo se considera un subconjunto aleatorio de todas las características disponibles. Esto evita que unas pocas características muy fuertes dominen todas las decisiones.

Proceso de clasificación

Cuando llega una nueva muestra de audio con sus características (formantes, MFCC, etc.), cada árbol del bosque realiza su propia clasificación. La predicción final se decide por mayoría:

- Si la mayoría de los árboles clasifican el audio como deepfake, la predicción final es deepfake.

- Si la mayoría lo clasifica como real, la predicción final es real.

Este mecanismo de votación reduce drásticamente la varianza del modelo: aunque un árbol individual pueda cometer errores, el consenso del bosque suele ser acertado.

Ventajas específicas para detección de deepfakes

- **Robustez ante ruido:** Las características de audio pueden contener variaciones por condiciones de grabación; Random Forest maneja bien estos datos ruidosos.
- **No requiere normalización:** A diferencia de redes neuronales o SVM, no es necesario escalar las características, lo que simplifica el pipeline.
- **Interpretabilidad:** El algoritmo proporciona una medida de importancia de cada característica, permitiendo identificar qué parámetros acústicos son más discriminativos entre voz real y sintética.
- **Resistencia al overfitting:** La combinación de bagging y aleatoriedad evita que el modelo memorice el conjunto de entrenamiento.

2.2.4. LightGBM

LightGBM (*Light Gradient Boosting Machine*) es un framework de gradient boosting desarrollado por Microsoft en 2017 [?] que optimiza el proceso de entrenamiento mediante innovaciones en la forma de construir los árboles y muestrear los datos. Mientras que Random Forest construye árboles de forma independiente y paralela, LightGBM los construye secuencialmente, donde cada nuevo árbol se especializa en corregir los errores de los anteriores.

Arquitectura y funcionamiento

El proceso de boosting funciona de la siguiente manera:

1. Se entrena un primer árbol simple que intenta clasificar las voces como reales o deepfake.
2. Se identifican los audios que este primer árbol clasificó incorrectamente.
3. Se entrena un segundo árbol enfocado específicamente en esos casos difíciles.
4. Se combinan ambos árboles para mejorar la precisión global.
5. El proceso se repite construyendo decenas o cientos de árboles, cada uno corrigiendo los residuos de los anteriores.

Innovaciones clave

- **Crecimiento por hoja (*leaf-wise*):** Los árboles tradicionales crecen nivel por nivel (balanceados). LightGBM, en cambio, crece expandiendo la hoja que más puede reducir el error en cada paso, lo que produce árboles más profundos en las regiones problemáticas y más eficientes en las fáciles.

- **Muestreo basado en gradientes (GOSS):** Durante el entrenamiento, LightGBM identifica qué instancias (audios) tienen mayor error y las prioriza. Las instancias con error pequeño se muestrean aleatoriamente para reducir el costo computacional. Esto permite entrenar con datasets grandes sin procesar toda la información redundante.
- **Agrupación de características (EFB):** Muchas características de audio raramente toman valores distintos de cero simultáneamente. LightGBM las agrupa para reducir la dimensionalidad, acelerando el entrenamiento sin pérdida de información significativa.

Proceso de clasificación

Cuando un nuevo audio llega para ser clasificado:

- Pasa secuencialmente por todos los árboles construidos.
- Cada árbol aporta una corrección o ajuste a la predicción anterior.
- La suma ponderada de todas las contribuciones produce la clasificación final (probabilidad de ser deepfake).

Ventajas específicas para detección de deepfakes

- **Alta precisión:** El enfoque secuencial de corrección de errores suele superar a Random Forest en problemas complejos como la detección de deepfakes.
- **Eficiencia computacional:** El muestreo inteligente permite entrenar con grandes volúmenes de audios en menos tiempo que otros métodos de boosting.
- **Manejo de desequilibrios:** Si existen desbalances entre clases, LightGBM incorpora mecanismos para ponderar las muestras adecuadamente.
- **Early stopping:** Se puede configurar para que detenga el entrenamiento cuando añadir más árboles no mejora la validación, evitando overfitting.

2.2.5. Support Vector Machines (SVM)

Las Máquinas de Vectores de Soporte (*Support Vector Machines*, SVM) son un conjunto de algoritmos de aprendizaje supervisado desarrollados por Vladimir Vapnik y su equipo en AT&T Bell Laboratories durante la década de 1990 [?]. Originalmente diseñadas para clasificación binaria, las SVM se han convertido en uno de los métodos más robustos y teóricamente fundamentados del aprendizaje automático.

Fundamento conceptual: El hiperplano óptimo

El objetivo fundamental de una SVM es encontrar un hiperplano que separe las dos clases (en nuestro caso, voces reales y voces generadas por IA) con el máximo margen posible. Para comprender este concepto, es útil visualizar un problema simple en dos dimensiones:

- Imaginemos que representamos cada audio como un punto en un plano, donde los ejes son dos características (por ejemplo, el primer formante y el primer MFCC).
- Las voces reales se agrupan en una región del plano y las generadas por IA en otra.
- Un hiperplano en dos dimensiones es simplemente una línea recta que intenta separar ambos grupos.

La clave de SVM reside en que no cualquier línea divisora es válida: el algoritmo busca aquella que maximiza la distancia (margen) entre la línea y los puntos más cercanos de cada clase. Estos puntos críticos, que “sostienen” el margen, reciben el nombre de **vectores de soporte** (*support vectors*), de donde el algoritmo toma su nombre.

Funcionamiento del algoritmo

El proceso de entrenamiento de una SVM puede entenderse en los siguientes pasos:

1. **Identificación de vectores de soporte:** El algoritmo examina todos los puntos de entrenamiento (audios con sus características) e identifica aquellos que son más difíciles de clasificar, es decir, los que están más cerca de la frontera entre clases.
2. **Maximización del margen:** A partir de estos vectores de soporte, la SVM calcula el hiperplano que maximiza la distancia a ambos lados. Este margen máximo proporciona la mejor capacidad de generalización: cuanto más ancho sea el margen, menor será la probabilidad de error al clasificar nuevos audios.
3. **Clasificación de nuevas muestras:** Una vez entrenado el modelo, para clasificar un nuevo audio, simplemente se calcula a qué lado del hiperplano se sitúa. Si cae en el lado de las voces reales, se clasifica como real; si cae en el lado de las generadas por IA, se clasifica como deepfake.

El truco del kernel: Separando lo inseparable

Uno de los mayores desafíos en la detección de deepfakes es que las voces reales y generadas por IA rara vez son linealmente separables en el espacio original de características. Es decir, no existe una línea recta (o hiperplano) que pueda separarlas perfectamente. Para abordar esta limitación, las SVM introducen el concepto de **kernel** o núcleo.

El *truco del kernel* (*kernel trick*) funciona de la siguiente manera conceptual:

- Los datos originales (no separables linealmente) se transforman implícitamente a un espacio de mayor dimensionalidad mediante una función matemática.
- En este nuevo espacio, es posible que los datos sí sean linealmente separables.
- La magia del kernel es que realiza esta transformación sin necesidad de calcular explícitamente las coordenadas en el espacio de alta dimensión, lo que sería computacionalmente prohibitivo.

Los kernels más comúnmente utilizados son:

- **Kernel lineal:** Asume que los datos son aproximadamente separables por un hiperplano. Útil cuando el número de características es muy grande.
- **Kernel polinomial:** Crea fronteras de decisión curvas mediante combinaciones polinómicas de las características originales.
- **Kernel RBF (*Radial Basis Function*):** Es el más utilizado en problemas de audio. Proyecta los datos a un espacio de dimensionalidad infinita, permitiendo fronteras de decisión altamente complejas y no lineales. Se basa en la similitud entre muestras medida por distancia euclidiana:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

donde γ controla la influencia de cada muestra: un valor pequeño considera vecindarios amplios (frontera más suave), mientras que un valor grande se enfoca en vecindarios reducidos (frontera más ajustada a los datos).

Parámetros críticos

El rendimiento de una SVM depende fundamentalmente de dos parámetros:

- **C (parámetro de regularización):** Controla el equilibrio entre tener un margen amplio y clasificar correctamente los puntos de entrenamiento. Un valor bajo de C busca un margen grande aunque se cometan algunos errores (modelo más generalista). Un valor alto de C penaliza fuertemente los errores, ajustándose más a los datos de entrenamiento (riesgo de overfitting).
- **γ (gamma):** Específico del kernel RBF, define hasta qué punto llega la influencia de un solo punto de entrenamiento. Valores bajos significan influencia de largo alcance (frontera más suave), mientras que valores altos hacen que cada punto tenga influencia muy local (frontera más compleja).

Ventajas específicas para detección de deepfakes

- **Fundamento teórico sólido:** A diferencia de otros métodos heurísticos, SVM está basada en la teoría de optimización convexa, lo que garantiza que la solución encontrada es óptima (máximo margen global).
- **Eficaz en espacios de alta dimensión:** Las características de audio pueden generar vectores de gran dimensionalidad (decenas o cientos de coeficientes). SVM maneja bien esta situación sin colapsar por la maldición de la dimensionalidad.
- **Robustez ante overfitting:** La maximización del margen proporciona una regularización natural que mejora la generalización, crucial cuando se trabaja con datasets limitados de voces deepfake.

- **Versatilidad mediante kernels:** La capacidad de elegir diferentes kernels permite adaptar el modelo a la estructura no lineal de los datos de audio sin necesidad de transformaciones manuales.
- **Interpretabilidad limitada pero útil:** Aunque no tan interpretable como los árboles de decisión, el análisis de los vectores de soporte puede revelar qué audios son más representativos o ambiguos en la frontera de decisión.

2.3. Modelos de explicación

Los modelos XAI (eXplainable Artificial Intelligence) han cobrado mucha importancia últimamente y más en términos de modelos de clasificación automáticos debido a la necesidad de saber el por qué se clasifican en un sitio u otro cada elemento, en nuestro caso cada audio.

Dos modelos muy utilizados son el modelo SHAP (SHapley Additive exPlanations) y el modelo LIME (Local Interpretable Model-agnostic Explanations), que se utilizan para el análisis de audio sintético. SHAP, como ya sabemos, cuantifica la contribución de cada característica a la predicción final y ayuda a visualizar qué bandas de frecuencia son indicativas de manipulación. Viendo que a menudo revela distorsiones en las zonas de alta frecuencia que el detector suele utilizar para clasificar una muestra como "fake". Por otro lado, LIME, crea explicaciones locales perturbando la entrada y observando los cambios en la salida. También vimos que en forense de audio, LIME se utiliza para generar máscaras sobre espectrogramas, señalando los milisegundos específicos donde se detectan inconsistencias fónicas.

Además de estos dos modelos, el modelo de Grad-CAM (Gradient-weighted Class Activation Mapping) se usa mucho junto con los modelos basados en CNN, ya que proporciona visualizaciones espaciales de las áreas del espectrograma que activan la decisión del modelo. Esto es importante para identificar si el modelo se está enfocando en anomalías espectrales o en ruido de fondo, por ejemplo.

Por tanto, como hemos visto lo más utilizado y útil son técnicas independientes del modelo, como SHAP y LIME, que muestran que características han servido para que el modelo clasifique un audio como real.^o "fake", y complementariamente para modelos específicos de CNN que usan espectrogramas, se usa Grad-CAM, que muestra que zonas del espectrograma influyen más en la decisión del modelo.

3. Metodología

3.1. Dataset

Para encontrar un dataset público que tuviera voces tanto reales como clonadas y que además fueran por pares (i.e. un audio de voz real cuya transcripción es "Hoy he comido en un restaurante chino" y un audio clonando dicha voz diciendo lo mismo) ha sido complicado. Hemos echado un ojo a bastantes datasets. Vamos a comentar los que hemos descartado y por qué.

1. HABLA [?]: Este dataset cuenta con una gran cantidad de audios tanto reales, como generados (con técnicas como TTS) y voces clonadas (con técnicas como Cycle-GAN). Lo hemos descartado principalmente por no contar con pares de audios, aunque tal vez podamos hacer pruebas con este dataset más adelante.
2. audioMNIST [?]: Este dataset cuenta con 30 mil audios de los números del 0 al 9 en inglés. Lo hemos descartado porque todos los audios son voces reales, por lo que no nos sirve para nuestra tarea.
3. ASVSpooof2021 [?]: Este dataset forma parte de una serie de desafíos internacionales organizados por la comunidad de verificación automática de audio cuyo objetivo es evaluar y mejorar los sistemas capaces de detectar voz manipulada. El dataset se compone de tres grupos de audio, todos ellos con voz real y generada mezclada, logical access con grabaciones transmitidas mediante canales de voz con efectos de codificación y transmisión, physical access con grabaciones de voz auténticas y ataques de reproducción en espacios físicos reales y speech deepfake con grabaciones generadas sintéticamente y comprimidas de formas distintas. Este dataset lo hemos descartado por no contar con pares de audio aunque se pueden rescatar algunos audios en inglés sobre todo para probar los modelos que probemos más adelante.
4. Kaggle

El dataset por el que nos hemos decantado se llama ODSS(<https://www.ai4europe.eu/research/ai-catalog/odss-open-dataset-synthetic-speech>). Hemos usado este conjunto de audios porque ya tiene creado un par para cada audio, lo que nos ahorra tiempo de generar nuestros propios audios. A esta ventaja, se le suma que ODSS ya contenía audios en Inglés y en Español, ambos idiomas son sumamente hablados por lo que es interesante centrarse en ellos.

Con todo esto mencionado, hemos tomado un subset de 100 audios de cada idioma. La mitad son audios naturales y la otra mitad audios generados. Más adelante probaremos a usar más cantidad de audios y compararemos el rendimiento.

3.2. Preprocesamiento

3.3. Extracción de características

Para la transcripción de los audios usamos modelos de vosk(<https://alphacephei.com/vosk/models>). Esto se debe a que tiene modelos para ambos idiomas que vamos a usar y además, 2 modelos dentro de cada lenguaje. El que usamos para las pruebas es el small ya que es más rápido y por lo tanto más útil para esta primera fase pero dejamos en consideración el modelo full para futuras fases ya que tiene mayor precisión. Sobre esto, cabe mencionar que primero probamos el modelo whisper pero tras varias pruebas con errores y la incapacidad para hacer funcionar, lo descartamos y es cuando escogimos vosk para esta tarea.

3.4. Modelos de clasificación

3.5. Modelos de explicación